

# Техническа Документация

Конфигуриране, Структура и Архитектура на Dockerизирана PHP-MySQL Среда

## 1. Архитектурна структура на проекта

Проектът е структуриран на модулен принцип, разделяйки изходния сорс код на приложението от конфигурационните файлове на инфраструктурата. Това позволява лесна преносимост и независимост на жизнения цикъл на контейнерите.

```
my-project/
├─ docker-compose.yml      # Главен оркестрационен файл за услугите
├─ Dockerfile              # Инструкции за изграждане на Apache+PHP контейнера
├─ database.sql            # SQL скрипт за първоначално импортиране на базата
├─ src/                   # Директория с PHP сорс кода на уеб сайта
│   ├── config.php        # Конфигурация за PDO връзка с базата данни
│   ├── index.php         # Главна страница
│   └─ [други PHP файлове] # Бизнес логика (login.php, posts.php и др.)
```

## 2. Компоненти на системата и тяхната работа

Системата се състои от два основни компонента (контейнеризирани услуги), които работят в синхрон:

| Компонент (Услуга) | Технологичен стек | Функционално описание и роля                                                                                                                                                           |
|--------------------|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Уеб Сървър ( web ) | PHP 8.2 + Apache  | Обработка входящите HTTP заявки от потребителите на порт 8080. Изпълнява PHP скриптовете от папка <code>src/</code> и генерира динамично HTML съдържание.                              |
| База Данни ( db )  | MySQL 8.0         | Съхранява персистентните данни на приложението. Капсулира релационните таблици <code>users</code> и <code>posts</code> и обслужва заявките от PHP приложението на вътрешния порт 3306. |

## Детайли за Dockerfile (Потребителски образ за web)

Тъй като официалният базово предоставен образ `php:8.2-apache` не съдържа разширения за работа с MySQL, се използва потребителски `Dockerfile` :

- Базиран на стабилна Linux среда с инсталиран Apache и PHP 8.2.
- Чрез инструмента `docker-php-ext-install` се компилират и добавят драйверите `pdo` и `pdo_mysql` , необходими за осъществяването на сигурна връзка с базата.

## 3. Комуникация и мрежова свързаност между услугите

Комуникацията е реализирана изцяло чрез вградената в Docker Compose изолация на виртуални мрежи:

- **Интра-контейнерна комуникация (DNS резолюция):** Вместо използването на статични IP адреси или нестабилния `localhost` , контейнерите се свързват чрез името на съответната услуга. Docker Compose автоматично конфигурира DNS сървър, благодарение на който PHP приложението намира базата данни на адрес `host=db` на вътрешния порт `3306` .
- **Изолация на портове:** Уеб контейнерът излага порт `8080` към реалния компютър на потребителя. Контейнерът на MySQL излага порт `3307` навън за административни цели (напр. връзка с MySQL Workbench), но вътрешната комуникация между PHP и MySQL остава защитена на порт `3306` .
- **Синхронизация на стартирането ( `depends_on` ):** Зададена е зависимост, която гарантира, че уеб сървърът няма да се вдигне преди контейнерът на базата данни да е инициализиран.

## 4. Изграждане, стартиране и първоначално импортиране

Целият процес по изграждане и пускане на инфраструктурата е автоматизиран и се извършва в терминала в коренната директория на проекта.

### Стъпка 1: Първоначално стартиране и изграждане (Build)

За компилиране на драйверите от `Dockerfile` и вдигане на двете услуги в демънизиран режим (на заден план), се изпълнява командата:

```
docker compose up -d --build
```

## Стъпка 2: Автоматично инициализиране на схемата на базата данни

В секцията `volumes` на услугата `db` е конфигурирано мапиране към специалната системна папка на MySQL: `./database.sql:/docker-entrypoint-initdb.d/init.sql`. Това гарантира, че:

- При **първоначалното** изграждане на базата данни, MySQL автоматично изпълнява инструкциите в изчистения `database.sql` файл.
- Автоматично се създават таблиците `users` (за автентикация) и `posts` (за публикации).

### Полезни команди за управление по време на работа:

- `docker compose ps` - Показва текущото състояние и статус на контейнерите.
- `docker compose logs db` - Извежда логовете на базата данни в реално време (за проверка за успешен импорт).
- `docker compose down --volumes` - Спира контейнерите и изтрива старите токове, което позволява пълно повторно зануляване и нов импорт на базата.